

MINISTÉRIO DA DEFESA
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
(Real Academia de Artilharia, Fortificação e Desenho - 1792)
SEÇÃO DE ENGENHARIA ELÉTRICA (SE/3)



**PROJETO DE SISTEMAS EMBARCADOS PARA
ENGENHARIA ELÉTRICA**

TRABALHO 3 – Sistema de Medição de Posição de Joystick com ESP32 e
ThingSpeak

PROFESSOR: CAP GABRIEL DA CRUZ FONTENELLE

INTEGRANTES:

CAP **ROBSON NUNES RODRIGUES JÚNIOR**

1º TEN **DAVI DE OLIVEIRA RABELO**

RIO DE JANEIRO

2026

Sumário

1	Introdução	2
2	Objetivos	2
3	Descrição geral do sistema	2
4	Componentes utilizados	3
5	Configuração do canal no ThingSpeak	3
6	Funcionamento da transmissão de dados	5
7	Esquemático elétrico	6
8	Projeto da placa de circuito impresso	7
9	Instruções de uso	8
10	Dificuldades e soluções encontradas	10
11	Conclusão	10

1 Introdução

Este trabalho descreve o desenvolvimento de um sistema embarcado com ESP32 para leitura de dados analógicos de um joystick de dois eixos e envio das medições para a plataforma ThingSpeak. O objetivo central foi integrar sensoriamento, comunicação Wi-Fi e visualização de dados em nuvem, usando a Arduino IDE como ambiente de programação.

O sistema lê os eixos X e Y do joystick pelas entradas analógicas do ESP32 e também monitora o botão integrado ao módulo. Os valores são enviados periodicamente para um canal no ThingSpeak, permitindo acompanhar graficamente as variações em tempo real.

Como a montagem física em protoboard não foi feita em sala, o relatório cobre a configuração do canal, a lógica do programa, o código-fonte, o esquemático elétrico e o desenho da placa de circuito impresso.

2 Objetivos

- Implementar um sistema de aquisição de dados com ESP32 capaz de medir a posição de um joystick analógico e enviar os valores para a nuvem.
- Configurar um canal no ThingSpeak com campos para os eixos X, Y e para o botão do joystick.
- Escrever o código em C++/Arduino para conexão Wi-Fi, leitura analógica e envio dos dados via requisição HTTP.
- Desenvolver o esquemático elétrico e o layout de placa de circuito impresso em ferramenta CAD (EasyEDA).
- Documentar o funcionamento do sistema, as decisões de projeto e as instruções de uso.

3 Descrição geral do sistema

O projeto é dividido em três blocos: aquisição de sinais, processamento local e transmissão para a nuvem. Na aquisição, o joystick gera sinais analógicos proporcionais ao deslocamento nos eixos horizontal e vertical. No processamento, o ESP32 converte esses sinais pelo ADC interno. Na transmissão, o Wi-Fi do ESP32 envia os dados ao ThingSpeak.

O joystick tem dois potenciômetros internos, um para cada eixo. Quando a haste é movida, a resistência varia e a tensão lida pelo ESP32 muda. O ADC do ESP32 trabalha com resolução de 12 bits, o que dá uma faixa de 0 a 4095.

O botão funciona como entrada digital. Com `INPUT_PULLUP`, o pino fica em nível alto em repouso e cai para nível baixo quando pressionado.

4 Componentes utilizados

Os principais componentes são o ESP32 DevKit, um módulo joystick analógico de dois eixos e uma fonte de alimentação compatível. No desenvolvimento, foram usadas a Arduino IDE e a plataforma ThingSpeak.

Tabela 1: Componentes e ferramentas utilizados no projeto

Componente/Ferramenta	Função no projeto	Observação
ESP32 DevKit	Microcontrolador responsável pela leitura dos sinais e envio à nuvem	Possui Wi-Fi integrado
Joystick analógico 2 eixos	Sensor de entrada com saídas X, Y e botão	Fornecer sinais analógicos e digitais
Arduino IDE	Ambiente de programação	Utiliza C++/Arduino
ThingSpeak	Plataforma de IoT para visualização dos dados	Recebe dados via API HTTP
EasyEDA	Elaboração do esquemático e PCB	Usado na documentação do projeto

5 Configuração do canal no ThingSpeak

Primeiro, cria-se uma conta no ThingSpeak e um novo canal para receber os dados. Para este projeto, foram habilitados três campos: **Field 1** para o eixo X, **Field 2** para o eixo Y e **Field 3** para o estado do botão.

Depois de criar o canal, acessa-se a aba **API Keys** e copia-se a **Write API Key**, que é usada no código para autorizar o envio das medições. Essa chave substitui o texto `SUA_WRITE_API_KEY` no programa.

Com o canal configurado, cada requisição HTTP do ESP32 atualiza os gráficos dos campos definidos. Assim, a movimentação do joystick aparece como variação nos gráficos do ThingSpeak em tempo real.

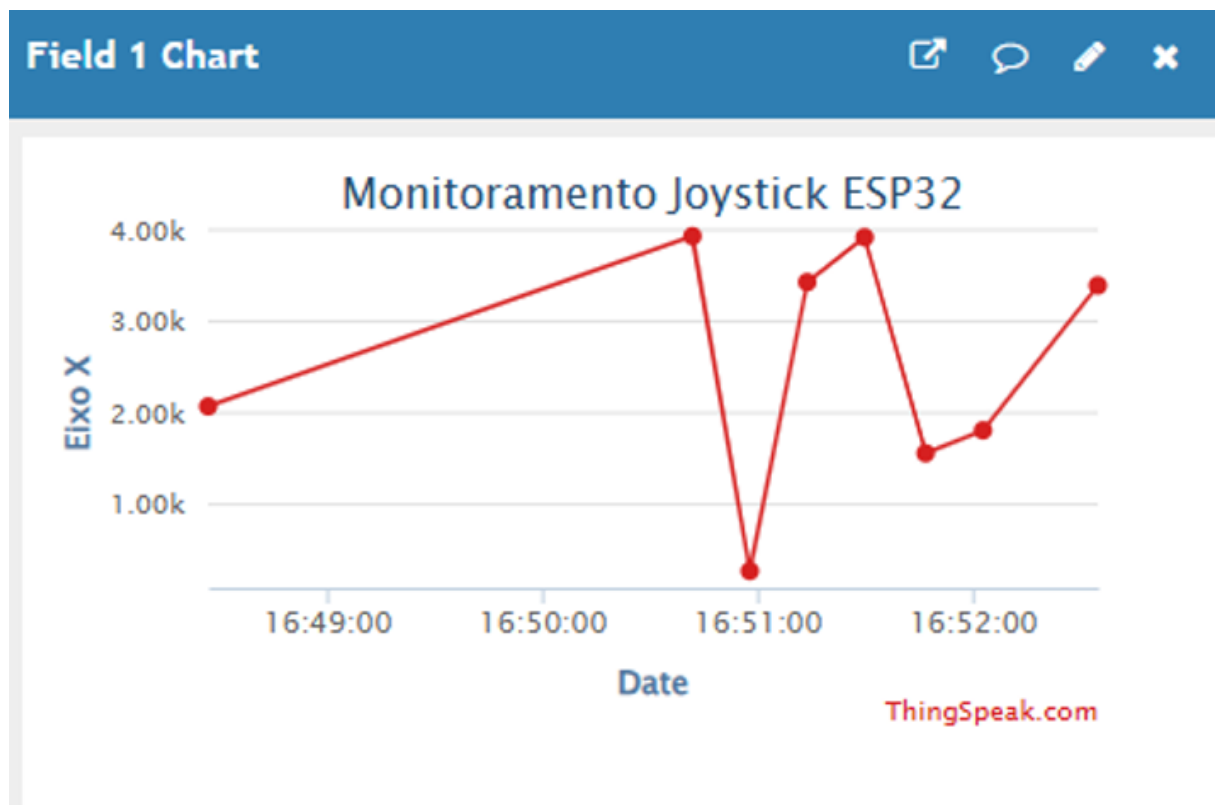


Figura 1: Gráfico do Field 1 no ThingSpeak.

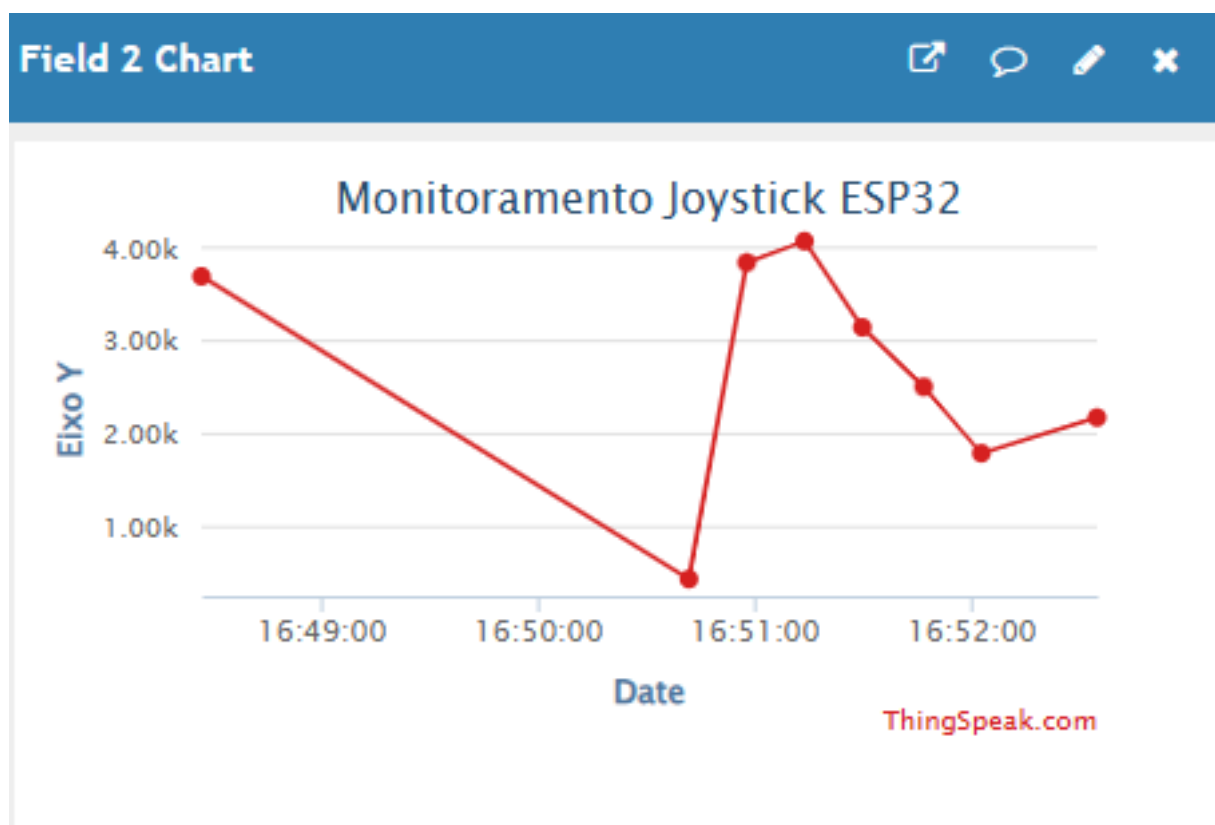


Figura 2: Gráfico do Field 2 no ThingSpeak.

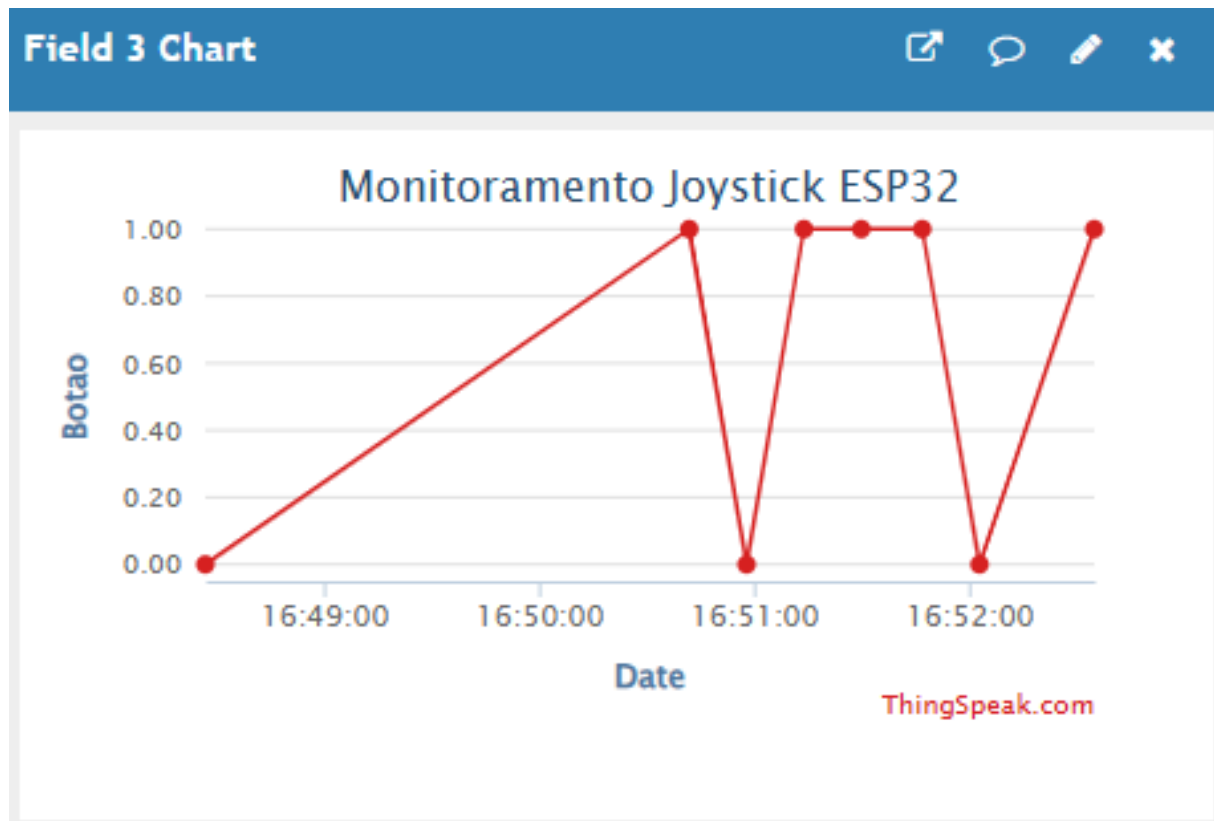


Figura 3: Gráfico do Field 3 no ThingSpeak.

6 Funcionamento da transmissão de dados

O envio dos dados é feito por uma requisição HTTP GET para o endereço de atualização do ThingSpeak. A URL carrega a chave de escrita do canal e os valores dos campos `field1`, `field2` e `field3`.

O fluxo do programa segue esta ordem: conectar ao Wi-Fi, ler os pinos do joystick, montar a URL de envio, executar o GET e esperar o intervalo mínimo antes da próxima atualização. O intervalo de 15 segundos foi necessário porque o ThingSpeak limita a frequência de atualizações em canais gratuitos.

Se a conexão Wi-Fi cair, o código tenta reconectar antes de continuar o envio, o que evita falhas durante o uso.

Fluxo simplificado:

1. Inicialização do ESP32 e abertura da comunicação serial.
2. Conexão à rede Wi-Fi configurada.
3. Leitura dos valores analógicos dos eixos X e Y.
4. Leitura digital do botão.
5. Montagem da URL com a Write API Key e os campos do canal.

6. Envio da requisição HTTP GET para o ThingSpeak.
7. Aguardo de 15 segundos antes da próxima medição.

7 Esquemático elétrico

O esquemático mostra a conexão entre o ESP32 e o joystick. A alimentação do módulo vai aos pinos 3,3 V e GND do ESP32; as saídas analógicas X e Y vão aos pinos GPIO 35 e GPIO 33, respectivamente; e o botão ao GPIO 32.

Esses pinos foram escolhidos por serem compatíveis com leitura ADC no ESP32 e por simplificar o código. O pull-up interno dispensa resistor externo no botão.

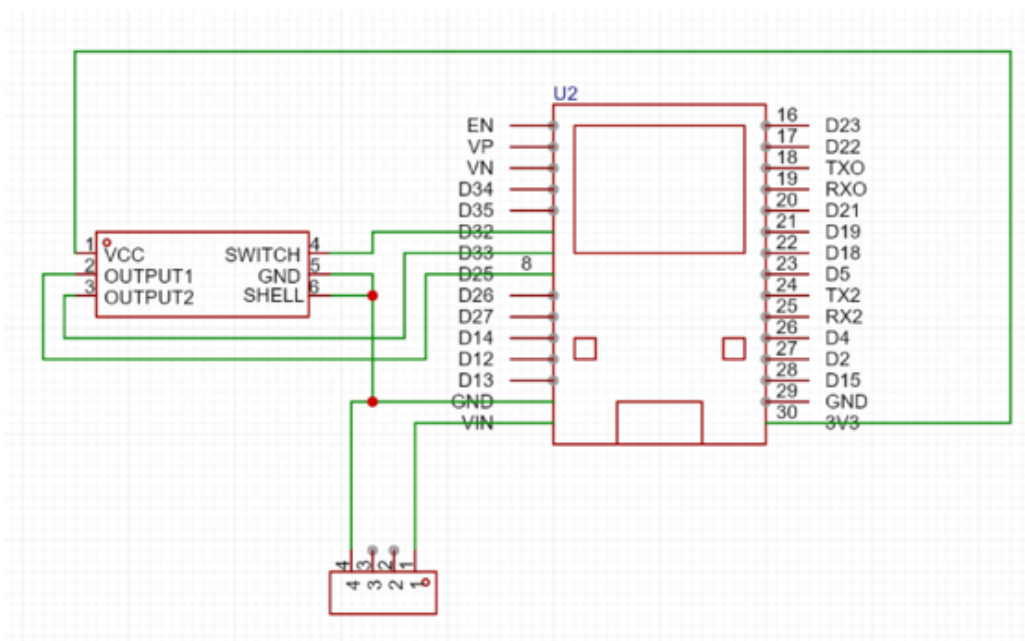


Figura 4: Esquemático elétrico do projeto.

Tabela 2: Ligações entre o joystick e o ESP32

Sinal	Pino do ESP32	Descrição
Eixo X	GPIO 35	Entrada analógica do eixo horizontal
Eixo Y	GPIO 33	Entrada analógica do eixo vertical
Botão	GPIO 32	Entrada digital com pull-up interno
VCC	3V3	Alimentação do joystick
GND	GND	Referência do circuito

8 Projeto da placa de circuito impresso

O objetivo da PCB é transformar o esquemático em um layout físico organizado para uma implementação permanente. No CAD, foram posicionados os conectores do joystick, os pinos do ESP32 e as trilhas que interligam os sinais.

O layout priorizou trilhas curtas, identificação clara dos pinos e separação adequada entre os sinais. Como o projeto tem baixa potência e poucos componentes, a placa ficou simples, facilitando a montagem.

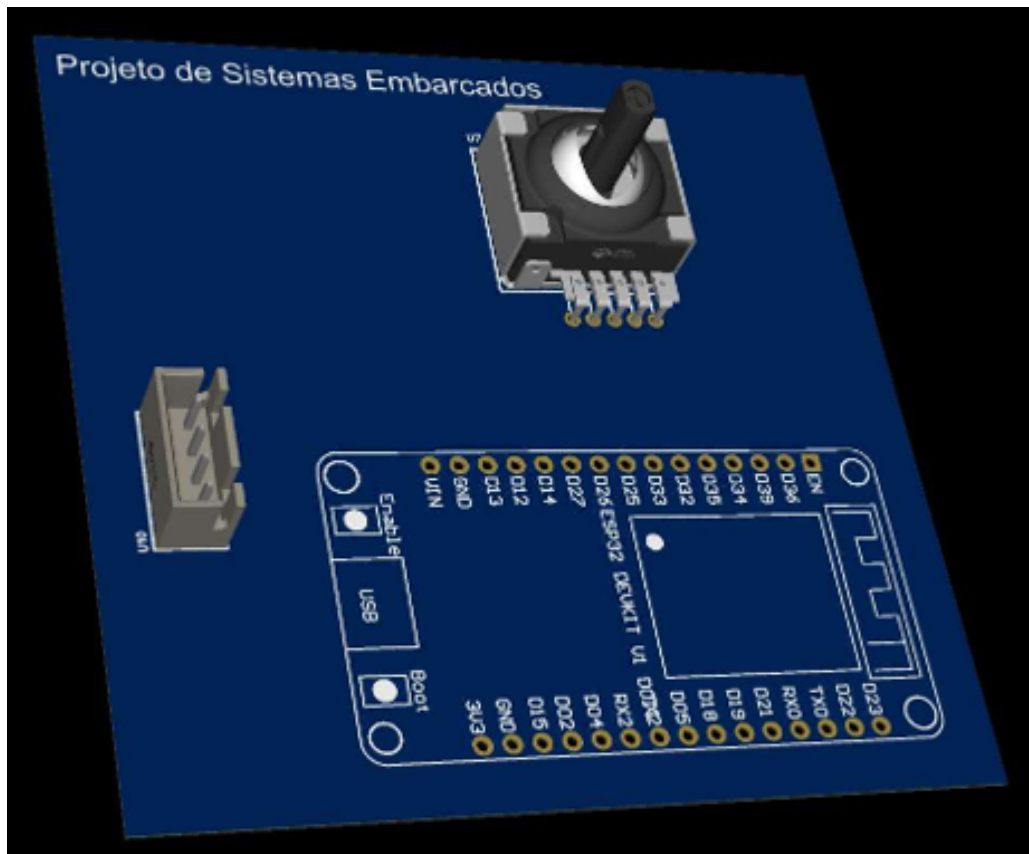


Figura 5: Visualização 3D da placa de circuito impresso.

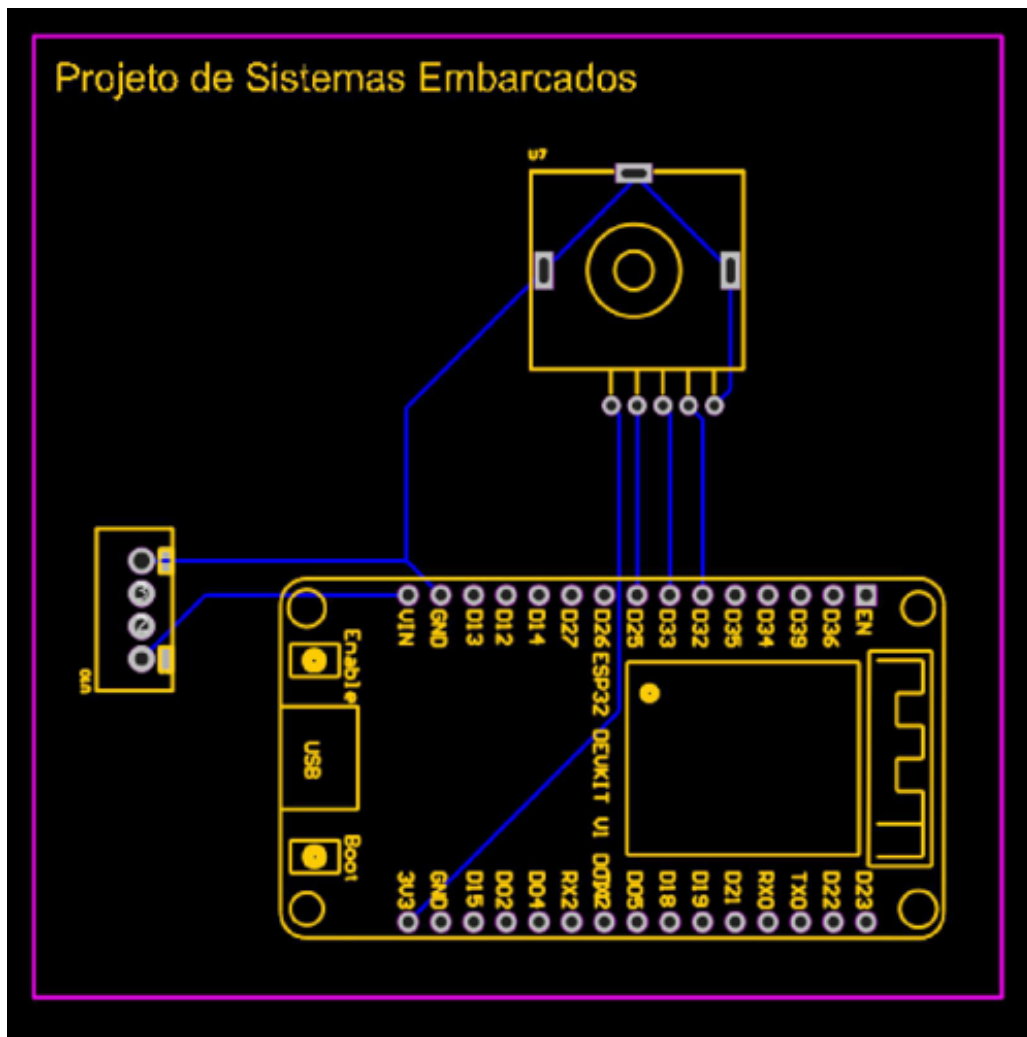


Figura 6: Layout da placa gerado a partir do esquemático.

9 Instruções de uso

Para usar o sistema, abrir o código na Arduino IDE, selecionar a placa ESP32, instalar as bibliotecas necessárias e preencher as credenciais do Wi-Fi e a chave do ThingSpeak no código.

Após gravar o programa no ESP32, abrir o Monitor Serial ajuda a confirmar se o Wi-Fi foi conectado e se os valores de X, Y e do botão estão sendo lidos. Em seguida, acessar o canal no ThingSpeak para ver os gráficos sendo atualizados.

Ao mover o joystick, os valores mudam conforme a posição da haste. Ao pressionar o botão, o campo correspondente vai para zero, o que aparece claramente no gráfico.

Código-fonte utilizado

```
#include <WiFi.h>
#include <HTTPClient.h>
```

```

// Credenciais da rede Wi-Fi
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// Endereco e chave do ThingSpeak
const char* server = "http://api.thingspeak.com/update";
const char* apiKey = "9FX5QMOPK7DCX2QE";

void setup() {
    Serial.begin(115200);

    WiFi.begin(ssid, password);
    Serial.println("Conectando ao Wi-Fi...");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println();
    Serial.println("Wi-Fi conectado!");
}

void loop() {
    // Leitura dos sensores
    int xValue = analogRead(35); // eixo X - GPIO 35
    int yValue = analogRead(33); // eixo Y - GPIO 33
    int buttonState = digitalRead(32); // botao - GPIO 32

    Serial.print("Eixo X: "); Serial.println(xValue);
    Serial.print("Eixo Y: "); Serial.println(yValue);
    Serial.print("Botao: "); Serial.println(buttonState);

    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;

        // Monta a URL com os valores dos campos
        String url = String(server) +
            "?api_key=" + apiKey +
            "&field1=" + String(xValue) +
            "&field2=" + String(yValue) +
            "&field3=" + String(buttonState);

        http.begin(url);
        int httpResponseCode = http.GET();

        if (httpResponseCode > 0) {
            Serial.print("Codigo HTTP: ");
            Serial.println(httpResponseCode);
        } else {
            Serial.print("Erro ao enviar: ");
            Serial.println(http.errorToString(httpResponseCode).c_str());
        }
    }
}

```

```
}

    http.end();
}

// Aguarda 15 s (limite minimo do ThingSpeak gratuito)
delay(15000);
}
```

10 Dificuldades e soluções encontradas

Um ponto de atenção foi garantir que os pinos usados fossem compatíveis com leitura analógica no ESP32. Isso foi resolvido escolhendo os GPIO 35 e GPIO 33, que são pinos ADC válidos.

Outro cuidado necessário foi respeitar o intervalo mínimo de atualização do ThingSpeak. Enviar dados muito rápido faz o servidor rejeitar as requisições, por isso foi adotado o atraso de 15 segundos.

Por fim, as credenciais de rede e a chave da API foram substituídas por valores genéricos no relatório e no repositório, mantendo os dados reais apenas no ambiente de teste.

11 Conclusão

O projeto juntou leitura analógica, programação embarcada e envio de dados para a nuvem num sistema funcional. O ESP32 foi uma escolha natural por já ter ADC e Wi-Fi embutidos, sem precisar de módulos externos.

Ver os dados no ThingSpeak em tempo real facilita bastante a validação: dá para confirmar que o joystick está sendo lido corretamente só olhando os gráficos. O esquemático e a PCB deixam o projeto mais organizado e fácil de reproduzir.

O trabalho cumpriu o que foi pedido e pode servir de ponto de partida para projetos maiores de monitoramento remoto e IoT.